

Введение в Python

Синтаксис, семантика, экосистема

Что такое Python

Python — скриптовый, мультипарадигменный (ООП, элементы ФП и др.) язык программирования общего назначения.

- простой синтаксис, низкий порог входа
- язык общего назначения
- разрабатывается сообществом, свободное ПО (спецификация и **CPython**)
- огромная экосистема библиотек для анализа данных, ML и AI
- основные области применения:
 - web-разработка (FastAPI, Django)
 - автоматизация и скрипты
 - анализ данных, машинное и глубокое обучение

2026: Python стабильно №1 в индексах TIOBE и PYPL, доминирует в Data Science и особенно в LLM/AI-разработке (PyTorch, JAX, Hugging Face).

Классификация языков программирования

Чтобы понять место Python, полезно взглянуть на общую классификацию:

Критерий	Варианты	Примеры
Способ выполнения	компилируемые / интерпретируемые / JIT / гибридные	C, Rust / Python, Ruby / Java, C#, JS(V8)
Типизация (когда)	статическая / динамическая	C++, Rust, Go / Python, JS
Типизация (строгость)	строгая / слабая	Python, Java / JS, PHP
Управление памятью	ручное / GC / владение (ownership)	C / Java, Python / Rust
Парадигма	императивные, ООП, функциональные, логические	C, Python, Haskell, Prolog

CPython — **интерпретируемый** (с байткод-компиляцией), **динамически и строго типизированный**, мультипарадигменный язык со сборкой мусора.

Компилируемые vs интерпретируемые vs JIT

Компилируемые — исходный код превращается в машинный код заранее (`gcc` , `rustc`):

- быстрый запуск и исполнение
- нет рантайма-посредника
- примеры: `C` , `C++` , `Rust` , `Go`

Интерпретируемые — код выполняется покомандно интерпретатором (`CPython` , `Ruby`):

- быстрый цикл разработки, кроссплатформенность
- медленнее "в лоб"
- примеры: `Python` , `Ruby` , `PHP`

JIT (Just-In-Time) — компиляция в машинный код во время выполнения:

- компромисс скорости и гибкости
- примеры: `Java` (JVM), `C#` (CLR), `JavaScript` (V8), `PyPy`

`CPython` на самом деле **гибридный**: код сначала компилируется в байт-код (`.pyc`), который затем интерпретируется виртуальной машиной.

Статическая и динамическая типизация

```
// Статическая типизация (C++)  
int a = 5;  
a = "hello"; // ошибка компиляции
```

```
# Динамическая типизация (Python)  
a = 5  
b = "hello" # Ok, тип определяется в рантайме
```

Проблема неявной динамики:

```
def add(a, b):  
    return a + b  
  
add(5, 10)      # 15  
add("5", "10")  # "510"
```

Частично решается через **type hints** (см. далее) и статические анализаторы (`mypy`, `pyright`).

Строгая и слабая типизация

```
# Строгая типизация – Python не молчит про несовместимые типы  
a = "10"  
b = a + 5 # TypeError
```

```
// Слабая типизация – JavaScript тихо приводит типы  
let a = "10";  
a = a + 5; // "105" (неявное приведение)
```

Python — язык со **строгой** типизацией: неявных, "магических" приведений между несовместимыми типами почти нет.

Реализации Python

CPython

- эталонная реализация
- исторический `Global Interpreter Lock` (`GIL`)
- с 3.13+: экспериментальная сборка **free-threaded** (без `GIL`, `PEP 703`), стабилизируется к 3.14–3.15
- с 3.13+: экспериментальный **JIT-компилятор** (`PEP 744` , copy-and-patch JIT)
- нативные расширения на C/C++/Rust

PyPy — `Just-in-time compilation` , обычно быстрее CPython, но не со всеми C-расширениями совместим

Jython — работает поверх `JVM`

GraalPython — поверх `GraalVM` , интероп с Java/JS

MicroPython / CircuitPython — облегчённые реализации для микроконтроллеров

Особенности (CPython)

- **строгая динамическая** типизация
- автоматическое управление памятью (подсчёт ссылок + циклический GC)

Явные преимущества:

- интерпретируемость, не нужно компилировать код (+/-)
- развитая стандартная библиотека
- огромное количество сторонних библиотек (PyPI > 600 000 пакетов)
- простота, отлично подходит для быстрого прототипирования

Недостатки:

- проблемы с производительностью в отдельных сценариях (частично решаются JIT/free-threading)
- проблемы со статическим анализом (динамическая природа усложняет верификацию)
- исторические проблемы дизайна (переход Python 2 → 3)

Инфраструктура

- [Python Software Foundation](#)
- **Python 2** давно устарел и не поддерживается (EOL — 2020)
- [PyPI](#) — репозиторий пакетов
 - NB: следить за безопасностью зависимостей (supply-chain атаки)
- [PEP](#) — **Python Enhancement Proposal**
- Система версионирования:
 - текущая стабильная ветка (на `09.2026`) — `Python 3.14.x`; `Python 3.15` в релиз-кандидатах
 - расписание версий регламентируется [PEP 602](#) / [PEP 719](#) (по релизу в год, октябрь)
 - обычная поддержка — 2 года багфиксов + ~3 года security-патчей

Type hints и статическая проверка

С [PEP 484](#) (Python 3.5+) появились аннотации типов — синтаксический сахар, не влияющий на выполнение, но помогающий инструментам:

```
def add(a: int, b: int) -> int:
    return a + b

from typing import Optional

def find_user(user_id: int) -> Optional[str]:
    ...

# Python 3.10+: короткий синтаксис union-типов
def find_user2(user_id: int) -> str | None:
    ...
```

Инструменты статического анализа: **mypy**, **pyright** (VSCode/Pylance), **pyre**. Позволяют ловить ошибки типов до запуска — компромисс между гибкостью Python и надёжностью статической типизации.

Python для научных вычислений и ML

Язык общего назначения, но с помощью сторонних библиотек — альтернатива `MATLAB`, `Mathematica`, `R`.

Преимущества:

- свободное ПО
- огромный выбор нативных библиотек — Python как `DSL` (Domain-specific language)
- пригоден для промышленного использования (production-ready)
- многие библиотеки написаны на C/C++/Rust (быстрое ядро + удобный интерфейс)
- инфраструктура для быстрого создания обёрток над низкоуровневыми библиотеками (`pybind11`, `cffi`, `Py03`)
- распределённые вычисления (`Dask`, `Ray`, `PySpark`)
- **де-факто стандарт** для обучения и инференса LLM (`PyTorch`, `JAX`, `Hugging Face Transformers`)

Недостатки:

- по эргономике для чистой статистики уступает специализированным языкам (например, R)
- ограничения интерпретатора CPython (постепенно снимаются JIT/free-threading)

Некоторые библиотеки

- [numpy](#) — линейная алгебра, случайные числа, базовые операции
- [scipy](#) — статистика, численные методы
- [polars](#) — быстрые dataframe на Rust-движке, альтернатива pandas
- [pandas](#) — dataframe-анализ в стиле R
- [jax](#) / [pytorch](#) — автодифференцирование, нейросети
- [transformers](#) — предобученные LLM и трансформеры (Hugging Face)
- [matplotlib](#) / [plotly](#) — визуализация
- [scikit-learn](#) — классический ML
- [catboost](#), [xgboost](#), [lightgbm](#) — градиентный бустинг

Установка Python

Linux:

- системный пакетный менеджер (apt, yum, pacman)

Windows, Linux, Mac OS:

- [Anaconda / Miniconda](#)
- **uv** — сверхбыстрый менеджер пакетов/окружений на Rust (де-факто новый стандарт 2025–2026)

Windows:

- WSL2

Docker:

- https://hub.docker.com/_/python/

Проверка версии:

```
$ python3 --version
$ uv python list    # современный способ управлять версиями
```

Командная строка Linux

Компьютерный терминал — аппаратное устройство для взаимодействия пользователя с компьютером (из истории вычислительной техники).

Эмулятор терминала, `tty` — программа, эмулирующая терминал средствами ОС.

Shell (оболочка ОС) — программа, предоставляющая интерфейс взаимодействия ОС с пользователем: графический или текстовый (**CLI**).

Примеры **Unix shell**'ов: `sh`, `csch`, `bash`, `zsh`, `fish`.

Windows: `PowerShell`, `CMD.EXE`, `WSL`.

Командные оболочки Linux

Обычно ввод командной строки выглядит так:

```
user@host:directory$
```

user — имя текущего пользователя

host — имя компьютера

directory — текущая директория

Некоторые команды в Unix shell

Узнать текущий шелл:

```
echo $SHELL
```

Работа с файловой системой:

```
cd, ls, cp, mv, rm, pwd, mkdir
```

Работа с файлами:

```
cat, head, tail, less, grep, wc
```

Конвейер команд (pipe):

```
cat a.txt | grep "Hello" | wc -l
```

REPL

REPL (read-eval-print loop) — «чтение-выполнение-вывод». Простая интерактивная консольная среда выполнения.

- **read** читает одно выражение из командной строки
- **eval** вычисляет значение выражения
- **print** печатает результат пользователю

Парадигма разработки:

- алгоритм разрабатывается небольшими порциями
- сразу тестируется
- отпадает необходимость в интерактивной отладке

Современные REPL: **IPython**, встроенный REPL Python 3.13+ (с подсветкой синтаксиса и многострочным редактированием).

Jupyter Notebooks

- физически файл с расширением `ipynb`
- ячейки:
 - текст, код или другой контент в зависимости от расширений
 - ячейку можно запустить и сразу получить результат
- графический/интерактивный вывод
- не только Python (`R`, `Julia`, `Kotlin`) — расширяемая поддержка через **ядра**
- **JupyterLab**: `pip install jupyterlab` → `jupyter lab`
- **Jupyter Notebook**: `pip install notebook` → `jupyter notebook`

Рабочая среда Jupyter

- WEB-браузер (**Jupyter Notebook** / **Jupyter Lab**)
 - ограничения при анализе кода
- **VSCode**
 - одна из самых удобных сред на 2026 год, особенно с расширением Jupyter + Pylance
 - встроенный **AI-автокомплит** (Copilot / Claude в редакторе)
- **PyCharm**
 - поддержка Jupyter только в Professional-версии
- **Google Colab** / **Kaggle** — ноутбуки прямо в браузере с GPU/TPU

Почему Jupyter Notebook?

- среда для интерактивного анализа данных и научных вычислений
- набор дополнений, возможность документирования
- богатые возможности визуализации
- интеграция с `pandas`, `polars`, `matplotlib` и другими библиотеками

Эффективная работа с Jupyter Notebook

- горячие клавиши (вставка/удаление/запуск ячеек)
- правила форматирования кода (`black` , `ruff format`)
- одна ячейка — одно логическое действие, один шаг обработки данных
- правила гигиены:
 - контроль версий (`nbdime` , `jupyterxtext` для diff-friendly ноутбуков)
 - постоянное документирование

Горячие клавиши

- `Ctrl+Enter` запустить ячейку
- `Ctrl+Shift+Enter` запустить ячейку и переместиться вниз
- `Esc+m` текстовая ячейка
- `Esc+y` ячейка с кодом
- `Ctrl+m+b` вставить новую ячейку за текущей
- `Ctrl+m+a` вставить новую ячейку перед текущей
- `Ctrl+m+x` удалить текущую ячейку

"Облачные" ноутбуки

- **Google Colaboratory** — <https://colab.research.google.com/>
 - доступ к GPU и TPU
 - интеграция с Google Drive
- **Kaggle Notebooks** — <https://www.kaggle.com/code>
- **Azure Machine Learning** — <https://ml.azure.com>
- **GitHub Codespaces** — <https://docs.github.com/en/codespaces>
- **Databricks Notebooks, Deepnote** — набирают популярность для команд